

main gate, you just gave your name; it could have been some criminal impersonating you. So, only after verifying that you are known to him does your friend let you in. That's the second step of verification.

Google's two-step verification

Two-step verification online works in a similar manner. Recently, Google introduced this feature as a part of its authentication process. Not many people might know about this, because it wasn't explicitly advertised. I myself came to know of it when I saw a friend of mine using this. A basic Google search will show you how this can be enabled for Google account(s). How it works is really simple. First, you enter your username and password. If the details provided are correct, you are taken to the second step of verification, where you are required to enter a randomly generated code delivered by Google Phone Verification System (PVS) to your registered cell phone number either through IVR or SMS. Upon entering this code, you are successfully authorised and taken to your account page. In case you do not have your phone with you, Google provides some backup codes, which can be downloaded from the *Account Settings* page.

However, remember one thing -- with two-step verification enabled, users cannot use third-party applications like mail clients, push mail, etc, linked to their Google account with the direct password. They have to generate a one-time password for each application from the *Accounts Security* page. Again, a quick Google search will show you the exact process.

Personally, I recommend that readers enable this feature, as it enhances the security of your account and makes it almost impossible to hack.

Pre-requisites for this tutorial

Now let's see how you can implement two-step verification in your Java applications. In order to cook this recipe, you will need the following ingredients: an SMS gateway account, Java SDK 6 or above, NetBeans 7.0 or above and the Java Mail API. I will cover some of these in brief below.

According to Wikipedia, an SMS gateway is "...a telecommunications network service that is used for sending or receiving Short Message Service (SMS) messages to or from a telecommunications network that supports SMS." Read more at http://en.wikipedia.org/wiki/SMS_gateway. For this exercise, I'll be using a Direct-to-SMSC (email to SMS) gateway, which means that you can send an SMS via email. To understand this more clearly, let's work through a simple exercise.

Browse to www.way2sms.com, which allows you to send free SMS, and create an account. Once it's done, log in and click on *Mail Alerts* and click *Activate* (if it's not activated). It'll create an ID for your account (something like 12345678@way2sms.com) to which you can forward mails, and you'll be notified of their arrival via SMS on

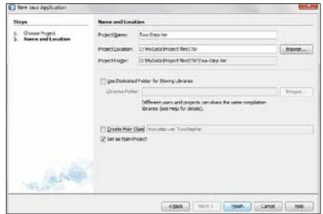


Figure 1: The new Java application window

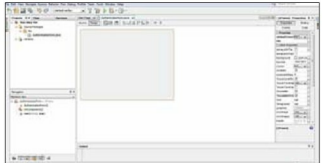


Figure 2: The Projects panel and Workspace after adding the form



Figure 3: Basic layout for the Authentication Form

your registered mobile number. You can also send an email to this ID, and the subject of that email will be displayed in the SMS you receive. You will be using this Way2SMS ID later, so note it down somewhere.

There are innumerable for-pay gateway services, but what other free gateway options have we got? Did you know that many mobile operators provide a free Direct-to-SMSC gateway? Just visit http://en.wikipedia.org/wiki/List_of_SMS_gateways and find out your operator's email-to-SMS-alert address.

Though free gateways will do for college projects or small applications, those developing a commercial application must opt for a paid service since there are 'no free lunches'! Free SMS gateways have certain limitations like: the SMS cannot be sent to DND subscribers, the SMS might be delivered late or not at all, a limitation to the number of SMSs that can be sent per day, and so on.

Java and NetBeans

The latest version (7.1) of the very popular NetBeans IDE is available at <http://netbeans.org/downloads/>. You may need to